AEGIS-1: A Hybrid Deep Learning System for Real-Time Tactical Drone Surveillance

Abstract

Modern battlefield surveillance necessitates robust, real-time threat detection systems that operate efficiently under constrained resource environments. This paper presents AEGIS-1, a cost-effective, hybrid deep-learning framework designed for military surveillance using unmanned aerial vehicles (UAVs). The system integrates a lightweight ESP32-CAM hardware module with a parallelized software architecture capable of running concurrent object detection models. By fusing the generalized detection capabilities of YOLOv8 with a custom-trained TensorFlow Lite (TFLite) model designated as "MIL-SCAN," the system achieves high-accuracy classification of specific military threats, including helicopters, military planes, tanks, vehicles, and missiles. The backend is orchestrated via Flask and optimized with multi-threading synchronization to ensure high frame rates without computational bottlenecking. The findings suggest that hybrid edge-cloud pipelines provide a scalable and reliable approach for tactical intelligence gathering.

1. Introduction

In recent years, the integration of Artificial Intelligence (AI) into military and tactical surveillance has transformed intelligence, surveillance, and reconnaissance (ISR) operations. Unmanned Aerial Vehicles (UAVs) equipped with computer vision capabilities are increasingly deployed to autonomously identify and track high-value targets. However, deploying computationally intensive deep learning models on resource-constrained drone platforms remains a significant challenge.

The AEGIS-1 project addresses this challenge by proposing a decoupled architecture where an edge-device (ESP32-CAM) acts as the video acquisition node, streaming MJPEG data to a more capable processing server. The server employs a hybrid detection strategy: a pre-trained YOLOv8 model for rapid, generalized object localization, and a bespoke TFLite model optimized for classifying distinct military hardware. This paper details the system architecture, hardware-software integration, and the multithreaded inference engine that enables real-time tactical analysis.

2. Related Work

Object detection in aerial imagery has been widely researched. Traditional approaches relied on handcrafted features (e.g., HOG, SIFT) coupled with SVM classifiers, which often struggled with variations in scale and orientation inherent to drone footage.

The advent of Convolutional Neural Networks (CNNs) revolutionized this field. Single-stage detectors like YOLO (You Only Look Once) and SSD (Single Shot MultiBox Detector) have become the standard for real-time applications due to their balance of speed and accuracy. However, deploying these large models directly on microcontrollers like the ESP32 is infeasible. Recent research has focused on model quantization and edge-computing frameworks, such as TensorFlow Lite Micro, to push inference closer to the sensor. AEGIS-1 builds upon these concepts by offloading the heavy inference to a centralized server while maintaining a lightweight sensor payload, thus maximizing drone flight time and minimizing hardware costs.

3. System Architecture and Methodology

3.1 Hardware Configuration

The sensor node of the AEGIS-1 system is built around the ESP32-CAM module, an ultra-small footprint camera module based on the ESP32 chip. It features an OV2640 camera capable of capturing JPEG frames. The module is configured to act as a web server, broadcasting a continuous MJPEG video stream over a local Wi-Fi network over HTTP.

3.2 Software Framework

The backend processing server is developed using Python and the Flask micro-framework. It acts as the central hub, ingesting the video stream, running the AI inference, and serving the processed tactical dashboard to the end-user via a web interface. MongoDB is integrated to persistently log detection events, enabling historical analysis of threat encounters.

3.3 Hybrid AI Detection Pipeline

The core innovation of AEGIS-1 lies in its dual-model inference engine:

- **Generalized Detection (YOLOv8):** A nano-version of YOLOv8 (yolov8n.pt) is utilized to provide a broad understanding of the scene. It operates at a high frame rate, identifying standard objects and providing contextual awareness.
- **Targeted Threat Classification (MIL-SCAN TFLite):** A custom-trained TensorFlow Lite model, designated "MIL-SCAN," runs in parallel. This model has been specifically trained on a curated dataset of military hardware and classifies detections into five categories: helicopter, military_plane, military_tank, military_vehicle, and missile. The TFLite model utilizes an input tensor shape optimized for specific resolution and operates with a strict confidence threshold (0.6) to minimize false positives.

3.4 Multi-threaded Processing and Synchronization

To ensure low latency, the system employs a highly parallelized multithreading architecture. Three dedicated threads handle the core operations:

1. 1. Capture Thread: Continuously reads the MJPEG stream, parses the JPEG byte boundaries, and updates the shared current frame.

2. 2. YOLO Inference Thread: Samples the current frame (utilizing frame-skipping techniques to maintain FPS) and executes YOLOv8 inference.
3. 3. TFLite Inference Thread: Concurrently samples the frame and executes the MIL-SCAN model.

To prevent race conditions and system crashes due to concurrent access to shared resources (e.g., the image frame or the AI execution context), strict synchronization mechanisms are implemented. threading.Lock() objects (ai_lock and frame_lock) guarantee thread-safe operations during matrix transformations and model invocation.

4. Implementation Details

4.1 Operating Modes

AEGIS-1 features two distinct operating modes to optimize resource utilization:

- **Live Mode:** The system actively pulls the stream from the ESP32-CAM and performs real-time hybrid inference.
- **Analysis Mode:** The live stream is suspended, and the system transitions into an offline state. In this mode, analysts can upload high-resolution imagery for deep-scan analysis. The YOLOv8 resolution is increased (e.g., imgsz=640), and strict deduplication logic is applied to the MIL-SCAN output to refine the intelligence report.

4.2 User Interface and HUD

The processed frames are composited with bounding boxes and labels using OpenCV. YOLOv8 detections are rendered in green, while high-value threats identified by the MIL-SCAN model are highlighted in red, providing a clear visual hierarchy for the operator. The feed is pushed to the client browser via a continuous multipart response (multipart/x-mixed-replace).

5. Results and Discussion

The implementation of the multi-threaded architecture successfully decoupled stream ingestion from inference, eliminating the "stop-and-go" lag commonly associated with sequential processing pipelines. The use of frame-skipping and memory locks ensured that the server CPU was not pinned at 100%, allowing for stable long-term operation.

The hybrid approach proved effective. YOLOv8 maintained spatial awareness, while the custom TFLite model demonstrated high precision in classifying the specific military targets. The integration of MongoDB allows for robust post-mission debriefing by logging the frequency and timestamps of detected threats.

6. Conclusion and Future Work

The AEGIS-1 project demonstrates the viability of using low-cost hardware combined with a sophisticated, multithreaded AI backend for tactical surveillance. By leveraging a hybrid

YOLOv8 and TFLite architecture, the system achieves real-time, accurate detection of military threats.

Future work will focus on integrating Edge TPU acceleration on the server side to further decrease inference latency. Additionally, migrating the custom model to a more advanced architecture like YOLO-NAS or implementing tracking algorithms (e.g., DeepSORT) could provide temporal continuity, assigning unique IDs to targets across multiple frames and enabling predictive trajectory analysis.

7. References

- 1. Redmon, J., & Farhadi, A. (2018). YOLOv3: An Incremental Improvement. arXiv preprint arXiv:1804.02767.
- 2. Jocher, G., et al. (2023). Ultralytics YOLOv8. GitHub Repository.
- 3. Abadi, M., et al. (2016). TensorFlow: Large-scale machine learning on heterogeneous systems.
- Grinberg, M. (2018). Flask Web Development: Developing Web Applications with Python. O'Reilly Media.